

JavaScript Random numbers, Control Statements and Arrays

Comprehensive Guide with Examples

JavaScript Math.random()

- The Math.random() function is used to generate a **pseudo-random number** between 0 (inclusive) and 1 (exclusive).
- This means it will always return a floating-point number greater than or equal to 0 but less than 1.

How Math.random() Works

- **Range:**
 - Returns values in the range [0, 1) (includes 0 but excludes 1).
- **Pseudo-Random:**
 - The numbers generated are deterministic and based on an algorithm, so they are not truly random.
- Example

```
let randomNumber = Math.random();  
console.log(randomNumber); // Example: 0.48257934957023
```

Generating a Random Number in a Range

- To generate random numbers in a custom range, use the following formula:

```
let randomInRange = Math.random() * (max - min) + min;
```

- Example: Random number between 5 and 10

```
let min = 5;  
let max = 10;  
let randomInRange = Math.random() * (max - min) + min;  
console.log(randomInRange); // Example: 7.345823
```

Generating a Random Integer

- To get an integer instead of a floating-point number, combine `Math.random()` with `Math.floor()` or `Math.ceil()`.
- **Example: Random integer between 1 and 100**

```
let randomInteger = Math.floor(Math.random() * 100) + 1;  
console.log(randomInteger); // Example: 42
```

Simulating a Dice Roll



- A dice has values from 1 to 6. Use the formula $\text{Math.floor}(\text{Math.random()} * 6) + 1$ to simulate a dice roll.

```
let diceRoll = Math.floor(Math.random() * 6) + 1;  
console.log(`You rolled: ${diceRoll}`); // Example: You rolled: 4
```

Practical Uses

- Games (e.g., dice rolls, random item generation)
- Shuffling elements
- Generating random test data
- Creating random keys or tokens

Using if-Else Conditionals

- The if-else statement allows you to execute different blocks of code based on conditions.
- if-else statements control the flow based on conditions.
- Syntax:

```
if (condition) {  
    // code if true  
} else {  
    // code if false  
}
```


Example

```
let age = 18;  
if (age >= 18) {  
    console.log('Eligible to vote');  
} else {  
    console.log('Not eligible to vote');  
}
```

If-Else: Real-World Example

- Example: Checking driving eligibility

```
let age = 16;  
if (age >= 18) {  
  console.log('You can drive.');} else {  
  console.log('You are too young to drive.');}
```

If-Else: Multiple Conditions

- Nested if-else checks multiple conditions sequentially.
- Example: Assigning grades

```
let score = 85;  
if (score >= 90) {  
    console.log('Grade: A');  
} else if (score >= 80) {  
    console.log('Grade: B');  
} else {  
    console.log('Grade: C');  
}
```

Comparators and Equality

- JavaScript supports various comparison operators:
- `==` (loose equality)
- `===` (strict equality)
- `!=` and `!==`
- `<`, `<=`, `>`, `>=`
- Example:

```
console.log(5 == '5');    // true
console.log(5 === '5');   // false
console.log(10 > 5);       // true
```

Strict vs Loose Equality

- `==` compares values regardless of type.
- `===` compares both value and type.
- Example:
- Loose Equality (==)

```
console.log(5 == '5'); // true
```

- Strict Equality (===)

```
console.log(5 === '5'); // false
```

Combining Comparators

- Use logical operators to combine conditions:
- `&&` (AND): Both conditions must be true.
- `||` (OR): At least one condition must be true.
- `!` (NOT): Negates the condition.
- Example: Voting eligibility

```
let age = 20;
let citizenship = 'USA';
if (age >= 18 && citizenship === 'USA') {
  console.log('You can vote.');
```

```
} else {
  console.log('You cannot vote.');
```

```
}
```

Challenge

- BMI Calculator Advanced (IF/ELSE)
- Leap Year exercises

Introduction to Arrays

- In JavaScript, an **array** is a data structure used to store a collection of values in a single variable
- Arrays can hold multiple data types (numbers, strings, objects, or even other arrays)
- Array are zero-indexed, meaning the first element has an index of 0.
- Creating Arrays:

```
// Using square brackets
```

```
let numbers = [1, 2, 3, 4];
```

```
// Using the Array constructor
```

```
let fruits = new Array('Apple', 'Banana', 'Cherry');
```


Accessing Array Elements

- Accessing Array Elements

```
let colors = ['Red', 'Green', 'Blue'];  
console.log(colors[0]); // Output: 'Red'  
console.log(colors[2]); // Output: 'Blue'
```

- Modifying Arrays

```
let animals = ['Dog', 'Cat', 'Bird'];  
animals[1] = 'Fish'; // Modifies the second element  
console.log(animals); // Output: ['Dog', 'Fish', 'Bird']
```

Common Array Methods

- **Adding/Removing/Searching Elements**
 - Includes(): Searching elements
 - push(element): Adds to the end of the array.
 - pop(): Removes the last element.
 - unshift(element): Adds to the beginning.
 - shift(): Removes the first element.

Searching in Array

- Use `includes()` method to search a specific element.
- It returns a boolean (true or false)
- Syntax:

```
array.includes(searchElement, fromIndex)
```

- **Parameters**
 - **searchElement**: The element to search for in the array.
 - **fromIndex** (*optional*): The index to start searching from.
 - Defaults to 0 if omitted.
 - If negative, it counts from the end of the array.

Examples

- Basic Example

```
let fruits = ['apple', 'banana', 'cherry'];  
console.log(fruits.includes('banana')); // true  
console.log(fruits.includes('grape')); // false
```

- **Case Sensitivity** The method is case-sensitive:

```
let words = ['Hello', 'world'];  
console.log(words.includes('hello')); // false
```

Includes(): Using fromIndex

```
let numbers = [1, 2, 3, 4, 5];  
console.log(numbers.includes(3, 2)); // true (starts search at index 2)  
console.log(numbers.includes(3, 3)); // false (starts search at index 3)  
console.log(numbers.includes(3, -3)); // true (starts search 3 from the end)
```

Adding and Removing Elements to Arrays

- Use `push()` to add elements to the end of an array.
- Use `unshift()` to add elements to the beginning.
- Example:

```
let arr = [1, 2, 3];  
arr.push(4); // [1, 2, 3, 4]  
arr.pop();   // [1, 2, 3]  
arr.unshift(0); // [0, 1, 2, 3]  
arr.shift();  // [1, 2, 3]
```

Challenges

- Random Array Elements challenge

Using While Loops

- A **while** loop in JavaScript repeatedly executes a block of code as long as a specified condition is true
- Syntax:

```
while (condition) {  
    // code to execute  
}
```


While Loops: How It Works:

- The **condition** is evaluated before each iteration.
- If the condition is **true**, the code inside the loop runs.
- This process continues until the condition becomes **false**.

Example 1: Counting from 1 to 5

```
let count = 1;

while (count <= 5) {
    console.log(count); // Output: 1 2 3 4 5
    count++;
}
```

Example 2: User Input Validation

```
let password = "";  
  
while (password !== "secret") {  
    password = prompt("Enter the correct password:");  
}  
  
console.log("Access granted!");
```

Key Points:

- **Infinite Loop Risk:** Be careful to ensure the condition will eventually become false, or you might create an infinite loop.
- **Break Statement:** Use `break;` to exit the loop early if needed.
- Challenges:
 - <https://www.99-bottles-of-beer.net/lyrics.html>

Using For Loops

- In JavaScript, **for loops** are used to run a block of code repeatedly based on a specified condition. They are useful when you know how many times you want the loop to run.
- Syntax of a For Loop

```
for (initialization; condition; increment) {  
    // Code to be executed  
}
```

- **Explanation:**
 - **Initialization:** Sets a starting value (usually a variable). It runs once before the loop starts.
 - **Condition:** Specifies the condition for running the loop. If the condition is true, the loop continues; if false, it stops.
 - **Increment/Decrement:** Updates the loop variable after each iteration.

Example 1: Counting Up

```
for (let i = 1; i <= 5; i++) {  
    console.log("Count:", i);  
}
```

- Output:

```
Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5
```

Example 3: Looping Through an Array

```
let fruits = ["Apple", "Banana", "Cherry"];  
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

- Output:

```
Apple  
Banana  
Cherry
```

Break and Continue Statements

- **break:** Stops the loop immediately.
- **continue:** Skips the current iteration and moves to the next.

Example

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue; // Skip the number 3  
  }  
  console.log(i);  
}
```

- Output:

```
1  
2  
4  
5
```

Challenges

- The Fibonacci Exercise